

Product case study · 2026

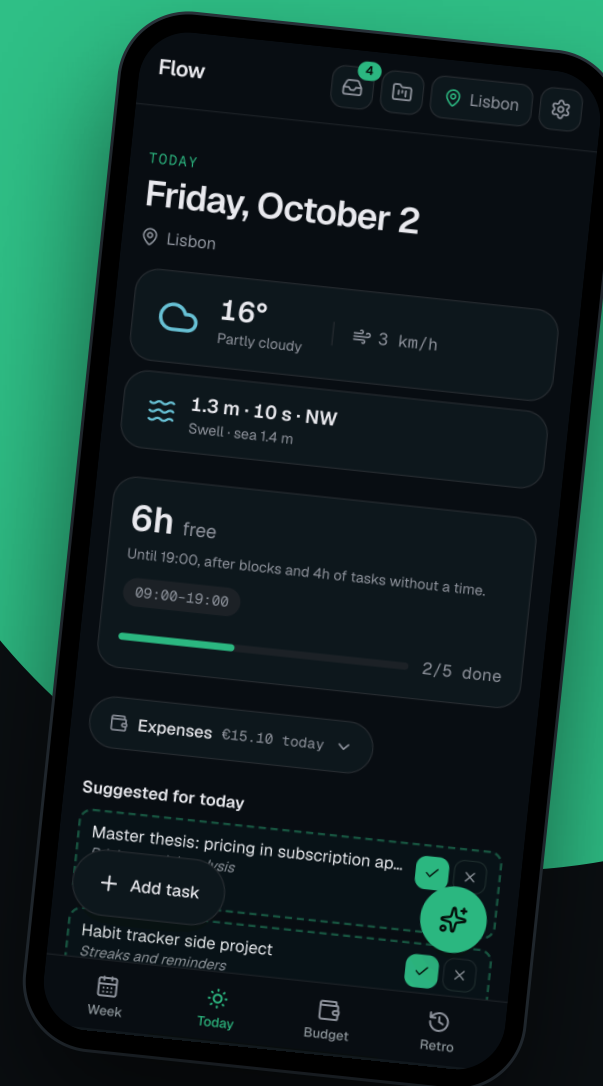
Flow.

Long-term work, made visible and doable
day by day. Plus a budget that finally makes
sense.

Designed and specified by me as Product Owner, implemented
with AI coding agents (Claude Code), shipped in 3 weeks.

● Real app · demo data

Source code is private · this deck shows the product, architecture and key decisions



Long-term work is invisible in day-to-day tools.

Oh

To-do lists only show what's due

A thesis or a side project never becomes "today's task" until it's already urgent.

?h free

Plans ignore real free time

Classes, routines, trips across time zones: nobody computes the hours that are actually left.

€ £ \$

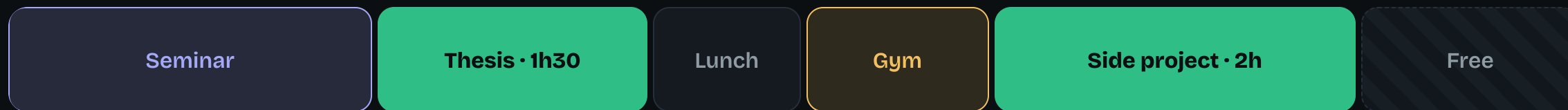
Money is all over the place

Expenses in several currencies and countries, and no clear picture of the month.

User #1: me. Master's student with side projects in parallel, always traveling. Every build tested daily on a real iPhone.

Treat projects as **minutes**, and pour them into real free time.

A Tuesday, 09:00 → 19:00



● Placed by the engine, nearest deadline first

Debt, with a date.

What doesn't fit becomes visible debt, with the date the project would slip.

Expenses in 2 taps.

Amount, category, done. Right currency, right country, right month.

AI proposes. I decide.

The assistant and booking emails only suggest. Nothing they propose is saved without a tap.

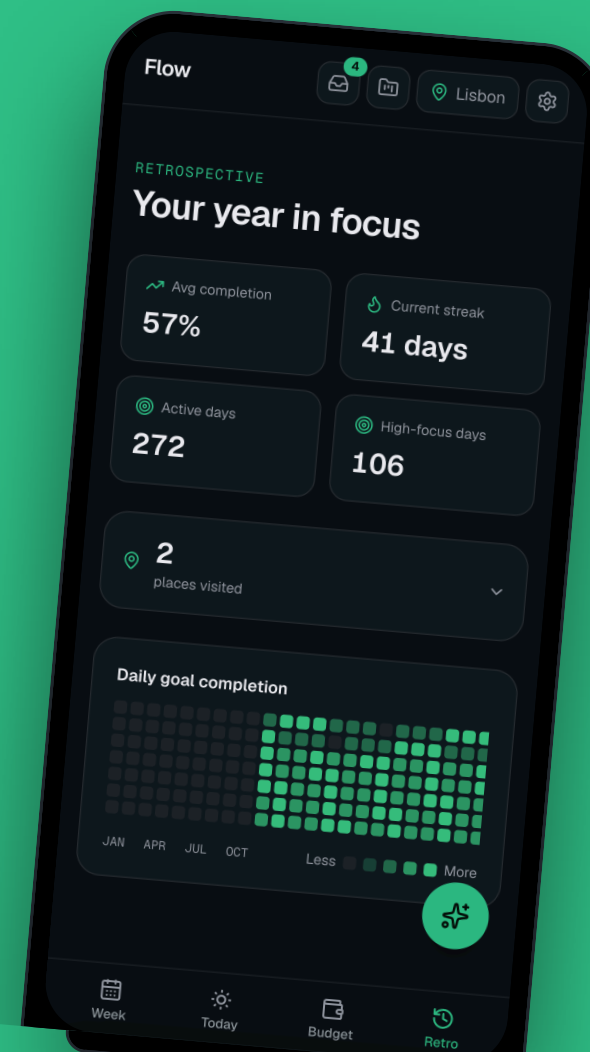
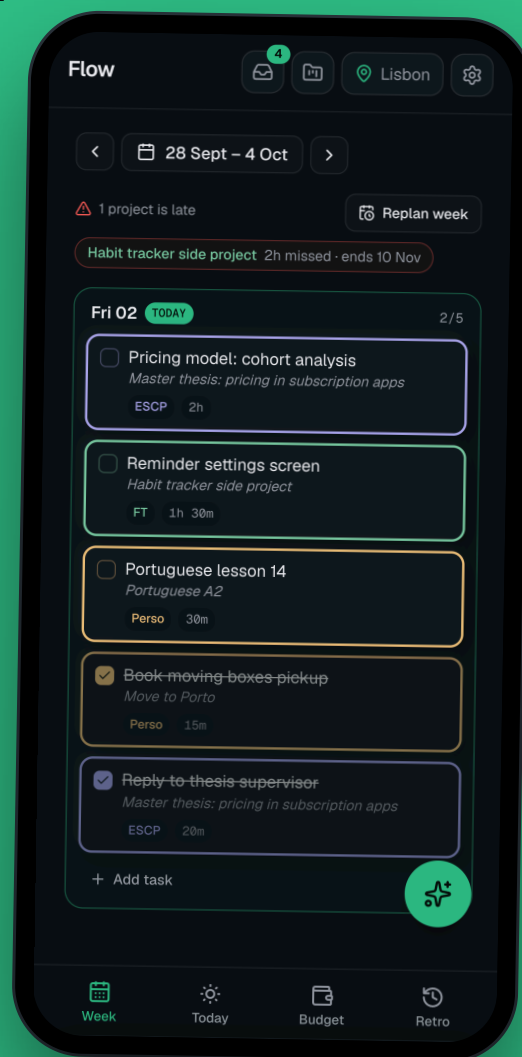
03 Plan the week

Today · Week · Retro

Retro: simulated one-year history to show the long-term view

• "6h free" is computed, not guessed

• Late projects get a slip date

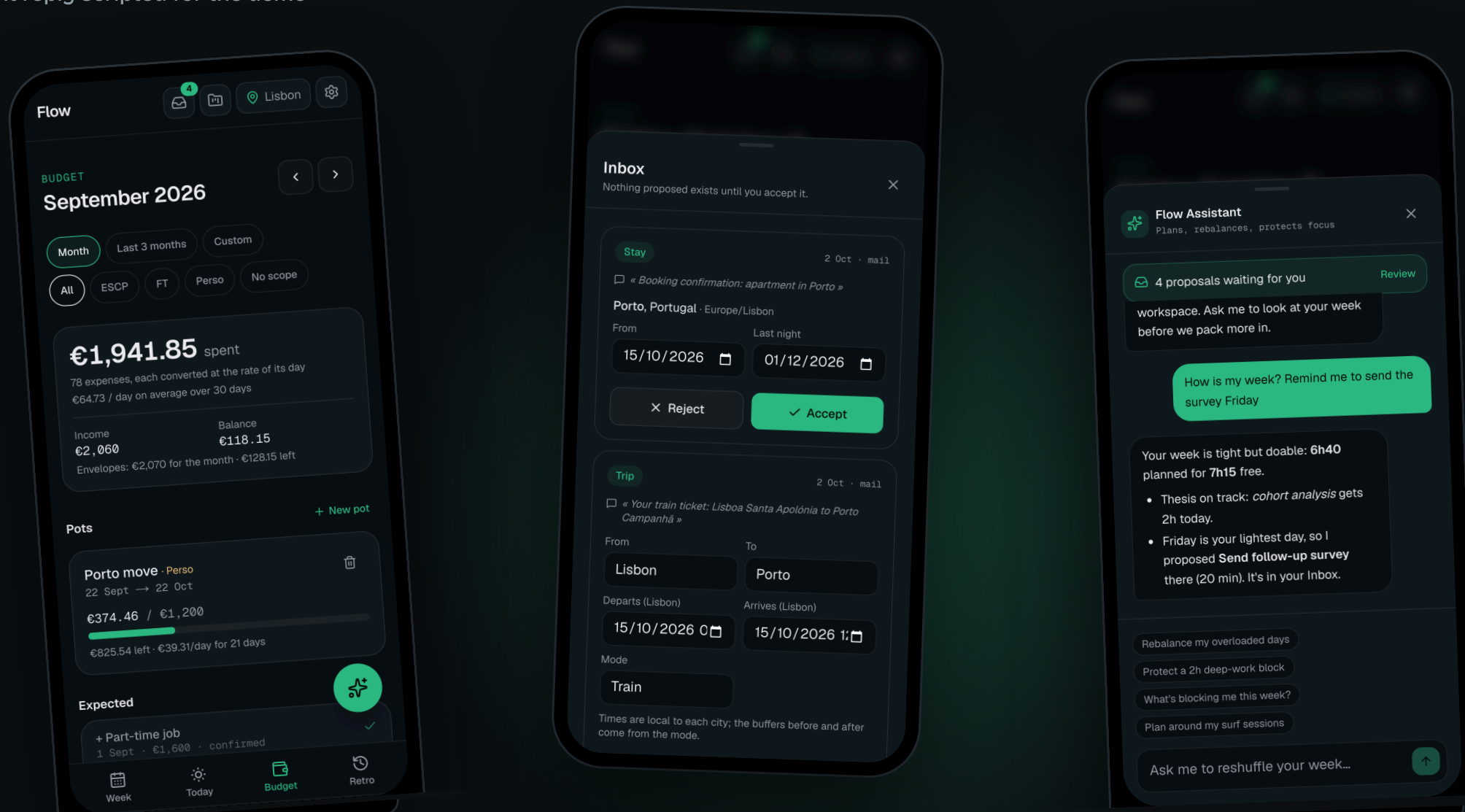


Budget · Inbox · Assistant

Assistant reply scripted for the demo

• Booking email → trip + stay + expense

• Created only after my OK



One app. AI on the server. Every row locked to its owner.



Every choice has a cost, written down.

AI

AI proposes, I validate

Every AI proposal goes through an Inbox.

Cost: one extra tap. **Win:** zero silent mistakes.

Planning

Capacity, not lists

Projects are minutes spread to their deadline. **Cost:** estimating hours. **Win:** delays get a date.

Capture

Emails without an agent

Structured data first, one AI call max, rules in code. **Cost:** less magic. **Win:** free and predictable.

Money

Rate frozen per day

Each expense stores its day's rate in €.

Cost: more data per row. **Win:** past months never drift.

Offline

Offline where it counts

Only 3 actions queue offline, with no duplicates. **Cost:** limited surface. **Win:** simple and reliable.

Process

Real environments, solo

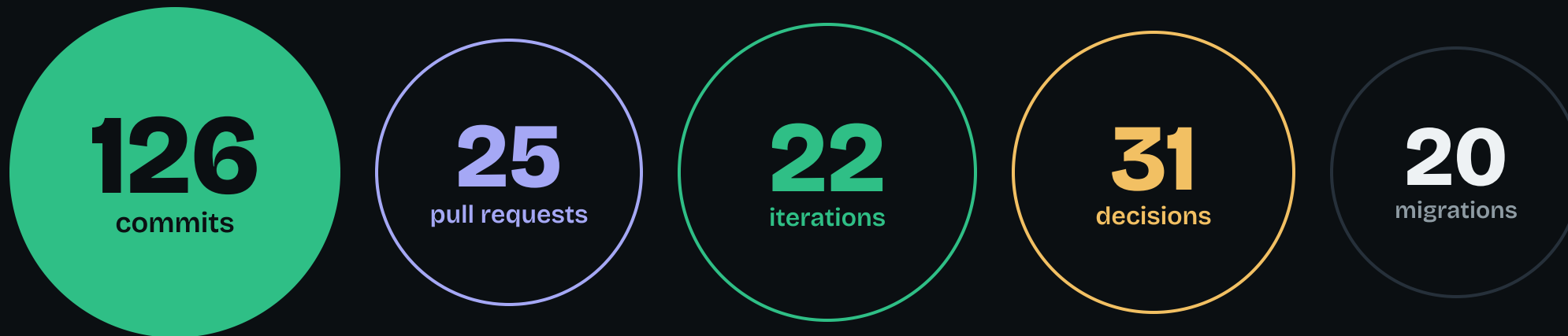
Preview per iteration, dev/prod split, I merge. **Cost:** more process. **Win:** ship daily, safely.

AI agents write the code. I own the product.

Me, as Product Owner: vision, roadmap cut into iterations, acceptance criteria, testing every build on my phone, the call to ship.

Claude Code, as the dev team: framed by a written contract: architecture rules, actions it may never take (secrets, production, unapproved migrations, out-of-scope refactors), and a "limits" log instead of silent scope creep.





What's next



Unified retro

One weekly score; time and money debt together.



Journaling

Daily notes that feed the retro and the assistant.



Anticipation agent

Spots conflicts (a deadline vs. a trip) before they happen, still as proposals.